

Data Structures Assignment 2

- 1- Implement Stack with LinkedList.
- 2- Implement Queue with LinkedList.
- 3- Implement Circular Queue.

4- Next Greater Element in Array (using stack data structure).

Given an array `arr[]` of integers, determine the Next Greater Element (NGE) for every element in the array, maintaining the order of appearance.

The Next Greater Element for an element x is defined as the first element to the right of x in the array that is strictly greater than x .

If no such element exists for an element, its Next Greater Element is -1.

Create method that take array and return array with next Greater Element

Examples:

Input: `arr[] = [1, 3, 2, 4]`

Output: `[3, 4, 4, -1]`

Explanation: The next larger element to 1 is 3, 3 is 4, 2 is 4 and for 4, since it doesn't exist, it is -1.

Input: `arr[] = [6, 8, 0, 1, 3]`

Output: `[8, -1, 1, 3, -1]`

Explanation: The next larger element to 6 is 8, for 8 there is no larger

elements hence it is -1, for 0 it is 1 , for 1 it is 3 and then for 3 there is no larger element on right and hence -1.

5- Reverse Queue

Given a queue `q[]`, reverse the queue so that the front element becomes the rear and the rear element becomes the front, while preserving the order of the remaining elements accordingly.

Create method that reverse queue in place

Examples:

Input: `q[] = [5, 10, 15, 20, 25]`

Output: `[25, 20, 15, 10, 5]`

Explanation: The original front 5 moves to the rear, and the rear 25 becomes the new front. All other elements follow the reversed order.

Input: `q[] = [1, 2, 3, 4, 5]`

Output: `[5, 4, 3, 2, 1]`

Explanation: The queue is reversed completely: 1 goes to the rear, and 5 moves to the front, with all intermediate elements rearranged accordingly.